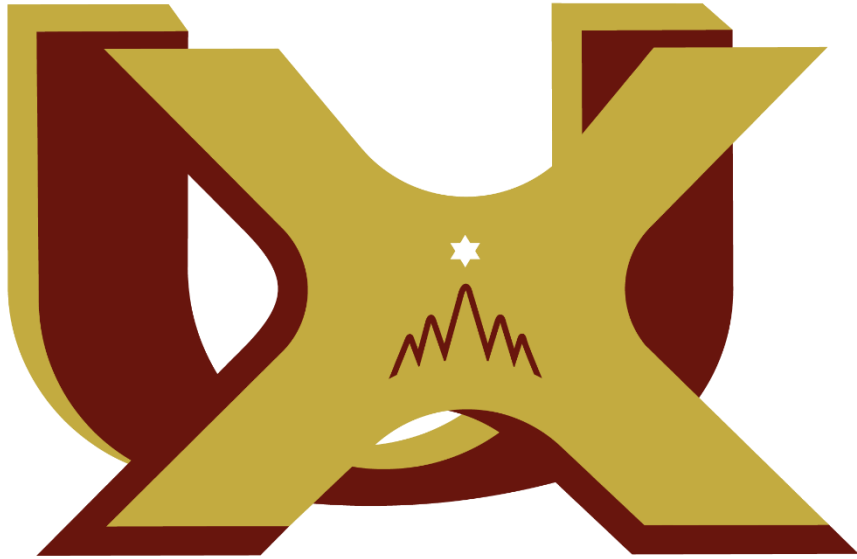


ACTIVIDAD 12. ORDENAMIENTO POR SELECCIÓN

Héctor Luis Garcia Marthen

Universidad Xalapa



UNIVERSIDAD DE XALAPA

• ————— •
Saber / Trascender

Parte 1.0

El objetivo de esta actividad es investigar y analizar el funcionamiento del **Método de ordenamiento por selección**, identificando sus características, etapas, ventajas, desventajas, complejidad y diferencias respecto a otros algoritmos de ordenamiento.

INVESTIGACIÓN Y PREGUNTAS

¿Qué es un algoritmo de ordenamiento?

Un algoritmo de ordenamiento es un conjunto de instrucciones paso a paso diseñado para reorganizar una colección de datos (como una lista o arreglo) en un orden específico, comúnmente numérico o lexicográfico (alfabético). Su objetivo es facilitar la búsqueda, optimizar la consulta de datos y estructurar la información de manera eficiente.

¿Qué es el ordenamiento por selección?

El ordenamiento por selección (Selection Sort) es un algoritmo de ordenamiento sencillo de tipo comparativo e in-place (no requiere memoria extra significativa). Opera dividiendo mentalmente la lista en dos partes: una sublista ya ordenada y otra sublista de elementos pendientes por ordenar.

¿Por qué recibe el nombre de "selección"?

Recibe este nombre porque en cada paso del proceso el algoritmo selecciona explícitamente el elemento más pequeño (o más grande, según el criterio) de la parte no ordenada y lo coloca en su posición definitiva.

¿Cuál es la idea principal del algoritmo?

La idea principal es recorrer repetidamente la sublista no ordenada para encontrar el elemento extremo (el mínimo en orden ascendente), moverlo al final de la sublista ordenada mediante un

intercambio, y reducir así en un elemento la sección no ordenada hasta que todo el arreglo esté completamente estructurado.

¿Cómo funciona el algoritmo paso a paso?

1. Se toma la primera posición de la parte no ordenada como referencia provisional del mínimo.
2. Se recorren los elementos restantes de la sublista no ordenada comparándolos con el mínimo provisional.
3. Si se encuentra un elemento menor, se actualiza la posición del elemento mínimo.
4. Una vez recorrido el resto de la lista, el mínimo definitivo encontrado se intercambia con el primer elemento de la sublista no ordenada.
5. El límite entre la parte ordenada y no ordenada se desplaza una posición a la derecha.
6. Se repiten los pasos 1 al 5 hasta que toda la lista quede ordenada.

¿Qué función cumple el elemento mínimo durante el proceso?

El **elemento mínimo** actúa como el objetivo o pivote de cada iteración. Representa el valor correcto que debe ocupar la primera posición disponible de la sublista no ordenada. Identificarlo asegura que los elementos se vayan colocando en su posición final absoluta uno por uno.

¿Cuándo se realiza un intercambio?

El intercambio (*swap*) se realiza **una sola vez al final de cada iteración del bucle exterior**, justo después de haber comparado todos los elementos restantes de la lista no ordenada. Si el índice del elemento mínimo hallado es diferente del índice inicial de la iteración, se intercambian sus posiciones.

¿El algoritmo necesita un arreglo auxiliar para ordenar los elementos?

No, es un algoritmo *in-place* (in situ). Realiza las permutaciones directamente sobre el arreglo original utilizando únicamente un par de variables auxiliares para almacenar índices y realizar el intercambio de valores. Su complejidad espacial es $O(1)$.

¿Puede utilizarse para ordenar de manera ascendente y descendente?

Sí. Para ordenar de manera **ascendente**, el algoritmo busca el elemento *mínimo* en cada iteración. Para ordenar de manera **descendente**, se invierte la condición de comparación para buscar el elemento *máximo* y colocarlo al principio de la sublista no ordenada.

¿Qué ocurre en cada iteración del algoritmo?

En cada iteración del bucle principal:

- Se asume temporalmente que el primer elemento no ordenado es el mínimo.
- Se escanea la sección no ordenada buscando si existe un valor menor.

- Se identifica la posición final del mínimo de esa iteración.
- Se realiza el intercambio de valores entre el mínimo encontrado y el elemento inicial de esa iteración.
- La longitud de la parte ordenada incrementa en 1 y la parte no ordenada se reduce en 1.

PARTE 1.1

TABLA COMPARATIVA

Aspecto	Burbuja (<i>Bubble Sort</i>)	Inserción (<i>Insertion Sort</i>)	Selección (<i>Selection Sort</i>)
Idea principal	Compara elementos adyacentes de forma iterativa y los intercambia si están desordenados, haciendo que los valores más grandes "floten" hacia el final.	Construye la lista ordenada elemento por elemento, tomando uno de la parte no ordenada e insertándolo en su posición correcta dentro de la sublista ya ordenada.	Busca el elemento mínimo (o máximo) en la sublista no ordenada y lo coloca al final de la sublista ordenada mediante un intercambio directo.
¿Cómo selecciona el elemento?	Compara de dos en dos los elementos	Toma el siguiente elemento no procesado (la	Recorre toda la sublista no ordenada buscando el índice

	adyacentes de manera secuencial de izquierda a derecha.	"clave") de la lista de forma lineal.	del valor con el peso mínimo (o máximo).
¿Cuándo realiza intercambios/movimientos?	Realiza intercambios continuos cada vez que detecta un par adyacente fuera de orden durante el recorrido.	Realiza desplazamientos continuos a la derecha de los elementos mayores para abrir espacio a la clave.	Realiza un solo intercambio por iteración (al final del recorrido de la sublista no ordenada).
Mejor caso	$O(n)$ (con optimización de bandera si la lista ya está ordenada)	$O(n)$ (si la lista ya está ordenada, solo realiza $n - 1$ comparaciones)	$O(n^2)$ (siempre debe recorrer el resto del arreglo para confirmar el mínimo)
Caso promedio	$O(n^2)$	$O(n^2)$	$O(n^2)$
Peor caso	$O(n^2)$	$O(n^2)$	$O(n^2)$
Complejidad temporal	$O(n^2)$ en general ($O(n)$ en el mejor caso optimizado)	$O(n^2)$ en general ($O(n)$ en el mejor caso)	$O(n^2)$ en todos los casos
Complejidad espacial	$O(1)$ (In-place)	$O(1)$ (In-place)	$O(1)$ (In-place)

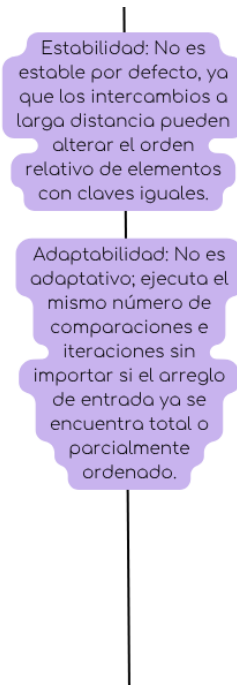
¿Es sencillo de implementar?	Sí , es de los algoritmos más simples de codificar y entender.	Sí , es intuitivo y fácil de implementar.	Sí , la estructura de bucles anidados es directa.
¿Es estable?	Sí <i>(mantiene el orden relativo de elementos con claves iguales)</i>	Sí <i>(mantiene el orden relativo si se implementa correctamente)</i>	No por defecto <i>(los intercambios a larga distancia pueden alterar el orden original de elementos iguales)</i>
Ventaja principal	Detección temprana de listas ya ordenadas (si se usa bandera); muy fácil de enseñar.	Gran eficiencia en listas pequeñas o conjuntos de datos casi ordenados ; es un algoritmo adaptativo.	Minimiza el número total de escrituras/intercambios en memoria (máximo $n - 1$ intercambios).
Desventaja principal	Ineficiente por el alto número de intercambios realizados en memoria.	Ineficiente para arreglos grandes debido a la cantidad de desplazamientos.	Rendimiento inflexible ($O(n^2)$) siempre, incluso si la lista ya está completamente ordenada).
Aplicaciones posibles	Fines educativos para introducir algoritmos de ordenamiento; listas	Ordenamiento de datos en tiempo real (streams) donde llegan datos uno por uno; arreglos casi	Sistemas donde el costo de escritura en memoria es extremadamente alto o limitado (ej. memorias EEPROM/Flash).

	muy pequeñas.	ordenados; base para <i>Timsort</i> .	
--	------------------	---	--

MAPA CONCEPTUAL

El mapa conceptual se realizó en canva





Fuentes de información consultadas

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Knuth, D. E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching (2nd ed.). Addison-Wesley Professional.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.

EJECUCIONES DEL PROGRAMA

N1

```
Pasada 1:  
1 3 7 4 2 9 8 6 5  
Pasada 2:  
1 2 7 4 3 9 8 6 5  
Pasada 3:  
1 2 3 4 7 9 8 6 5  
Pasada 4:  
1 2 3 4 7 9 8 6 5  
Pasada 5:  
1 2 3 4 5 9 8 6 7  
Pasada 6:  
1 2 3 4 5 6 8 9 7  
Pasada 7:  
1 2 3 4 5 6 7 9 8  
Pasada 8:  
1 2 3 4 5 6 7 8 9  
  
Comparaciones hechas: 36  
Intercambios hechos: 7  
PS C:\msys64\ucrt64\bin>
```

N2

```
PS C:\msys64\ucrt64\bin> & "C:\Users\hecto\OneDrive\Documents\Programas\Bubble Sort\Bubble Sort.exe" 1 2 3 4 5 6 7 8 9  
Pasada 1:  
1 2 3 4 5 6 7 8 9  
Pasada 2:  
1 2 3 4 5 6 7 8 9  
Pasada 3:  
1 2 3 4 5 6 7 8 9  
Pasada 4:  
1 2 3 4 5 6 7 8 9  
Pasada 5:  
1 2 3 4 5 6 7 8 9  
Pasada 6:  
1 2 3 4 5 6 7 8 9  
Pasada 7:  
1 2 3 4 5 6 7 8 9  
Pasada 8:  
1 2 3 4 5 6 7 8 9  
  
Comparaciones hechas: 36  
Intercambios hechos: 0  
PS C:\msys64\ucrt64\bin>
```

N3

```
Intercambios hechos: 0
● PS C:\msys64\ucrt64\bin> & "C:\Users\hecto\OneDrive\
Pasada 1:
1 8 7 6 5 4 3 2 9
Pasada 2:
1 2 7 6 5 4 3 8 9
Pasada 3:
1 2 3 6 5 4 7 8 9
Pasada 4:
1 2 3 4 5 6 7 8 9
Pasada 5:
1 2 3 4 5 6 7 8 9
Pasada 6:
1 2 3 4 5 6 7 8 9
Pasada 7:
1 2 3 4 5 6 7 8 9
Pasada 8:
1 2 3 4 5 6 7 8 9

Comparaciones hechas: 36
Intercambios hechos: 4
○ PS C:\msys64\ucrt64\bin> █
```

N4

```
● PS C:\msys64\ucrt64\bin> & "C:\Users\hecto\OneDrive\
Pasada 1:
1 2 3 5 4 6 7 8 9
Pasada 2:
1 2 3 5 4 6 7 8 9
Pasada 3:
1 2 3 5 4 6 7 8 9
Pasada 4:
1 2 3 4 5 6 7 8 9
Pasada 5:
1 2 3 4 5 6 7 8 9
Pasada 6:
1 2 3 4 5 6 7 8 9
Pasada 7:
1 2 3 4 5 6 7 8 9
Pasada 8:
1 2 3 4 5 6 7 8 9

Comparaciones hechas: 36
Intercambios hechos: 1
○ PS C:\msys64\ucrt64\bin> █
```

N5

```
Pasada 98:
11 27 33 39 51 62 68 71 73 88 93 96 116 118 132 147 149 156 174 182 184 194 197 203 213 224 225 245 251 263 268 278 285 29
6 309 315 326 334 341 359 367 377 382 395 403 421 438 451 459 476 481 514 519 527 529 542 547 556 564 583 591 596 604 608
612 634 638 653 681 683 690 697 705 714 719 721 727 746 749 762 765 772 778 811 813 824 829 847 856 864 875 879 903 905 93
2 936 946 957 998 987
Pasada 99:
11 27 33 39 51 62 68 71 73 88 93 96 116 118 132 147 149 156 174 182 184 194 197 203 213 224 225 245 251 263 268 278 285 29
6 309 315 326 334 341 359 367 377 382 395 403 421 438 451 459 476 481 514 519 527 529 542 547 556 564 583 591 596 604 608
612 634 638 653 681 683 690 697 705 714 719 721 727 746 749 762 765 772 778 811 813 824 829 847 856 864 875 879 903 905 93
2 936 946 957 987 998
Comparaciones hechas: 4950
Intercambios hechos: 96
PS C:\msys64\ucrt64\bin>
```

N6

```
Pasada 98:
9 21 41 43 52 57 68 72 81 92 98 105 119 128 137 146 156 164 177 182 186 191 215 224 235 242 253 264 275 291 298 308 314 31
7 326 333 346 352 354 371 387 394 413 425 438 453 462 478 488 501 516 521 528 537 549 561 574 590 598 607 611 614 623 629
643 645 659 667 681 682 694 704 708 726 731 735 755 761 769 781 782 793 803 815 827 839 846 869 875 884 886 899 912 918 94
7 954 957 973 983 996
Pasada 99:
9 21 41 43 52 57 68 72 81 92 98 105 119 128 137 146 156 164 177 182 186 191 215 224 235 242 253 264 275 291 298 308 314 31
7 326 333 346 352 354 371 387 394 413 425 438 453 462 478 488 501 516 521 528 537 549 561 574 590 598 607 611 614 623 629
643 645 659 667 681 682 694 704 708 726 731 735 755 761 769 781 782 793 803 815 827 839 846 869 875 884 886 899 912 918 94
7 954 957 973 983 996
Comparaciones hechas: 4950
Intercambios hechos: 92
PS C:\msys64\ucrt64\bin>
```

N7

```
Pasada 98:
22 27 36 44 47 58 61 65 73 84 96 98 113 119 126 134 145 152 157 173 174 186 189 203 218 247 251 256 263 267 286 294 315 31
7 324 328 335 341 354 369 371 398 402 427 428 439 453 462 476 491 517 529 532 540 548 556 581 583 590 603 607 608 621 634
638 647 671 684 694 695 709 714 716 725 729 756 761 765 773 779 789 803 812 817 827 845 849 862 875 881 883 905 916 918 93
4 937 941 953 972 996
Pasada 99:
22 27 36 44 47 58 61 65 73 84 96 98 113 119 126 134 145 152 157 173 174 186 189 203 218 247 251 256 263 267 286 294 315 31
7 324 328 335 341 354 369 371 398 402 427 428 439 453 462 476 491 517 529 532 540 548 556 581 583 590 603 607 608 621 634
638 647 671 684 694 695 709 714 716 725 729 756 761 765 773 779 789 803 812 817 827 845 849 862 875 881 883 905 916 918 93
4 937 941 953 972 996
Comparaciones hechas: 4950
Intercambios hechos: 95
PS C:\msys64\ucrt64\bin>
```